Cloud Computing

UNIT-3

Design Considerations for Cloud Applications

1. Scalability *(capacity to be changed in size)*

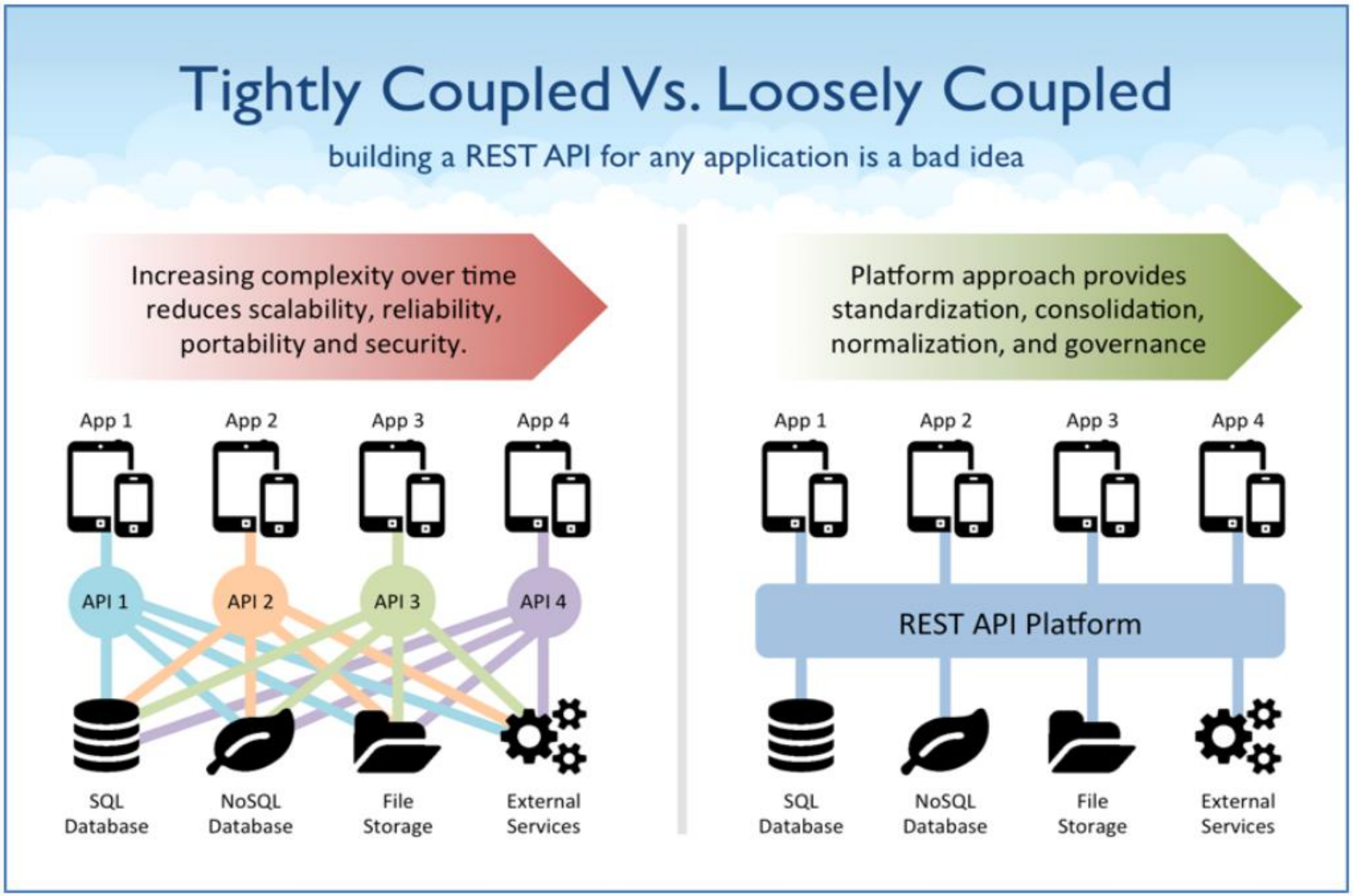
- Scalability is an important factor that **drives the application designer to move to cloud computing environments.**
- Building applications that **can serve millions of users without taking a hit on their performance** has always been challenging.
- With the growth of cloud computing applications designers can provision adequate resources to meet their workload levels.
- Traditional approaches were based on either over provisioning(equipment) of resources to handle the peak workload levels expected or provisioning based on average workload levels.
- To provide the benefits of cloud must implement the following design considerations.

1. Loose coupling of components
2. Asynchronous Communication
3. Stateless Design
4. Database choice and design

Design Considerations for Cloud Applications

1 . Loose coupling of components

- Traditional Application design Methodologies with tightly coupled application components, limit the scalability.
- **Tight Coupling is the idea of binding resources to specific purposes and functions.**
- **By designing loosely coupled components it is possible to scale each component independently.**

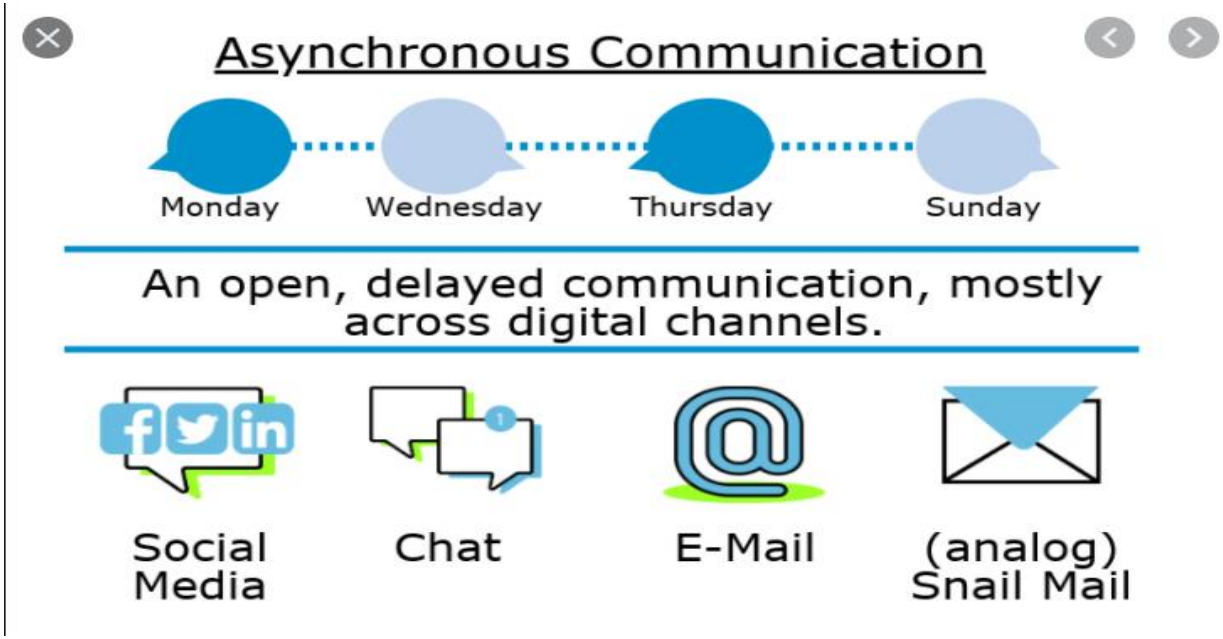


loosely coupled components it is possible to scale each component independently

Design Considerations for Cloud Applications

2. Asynchronous Communication

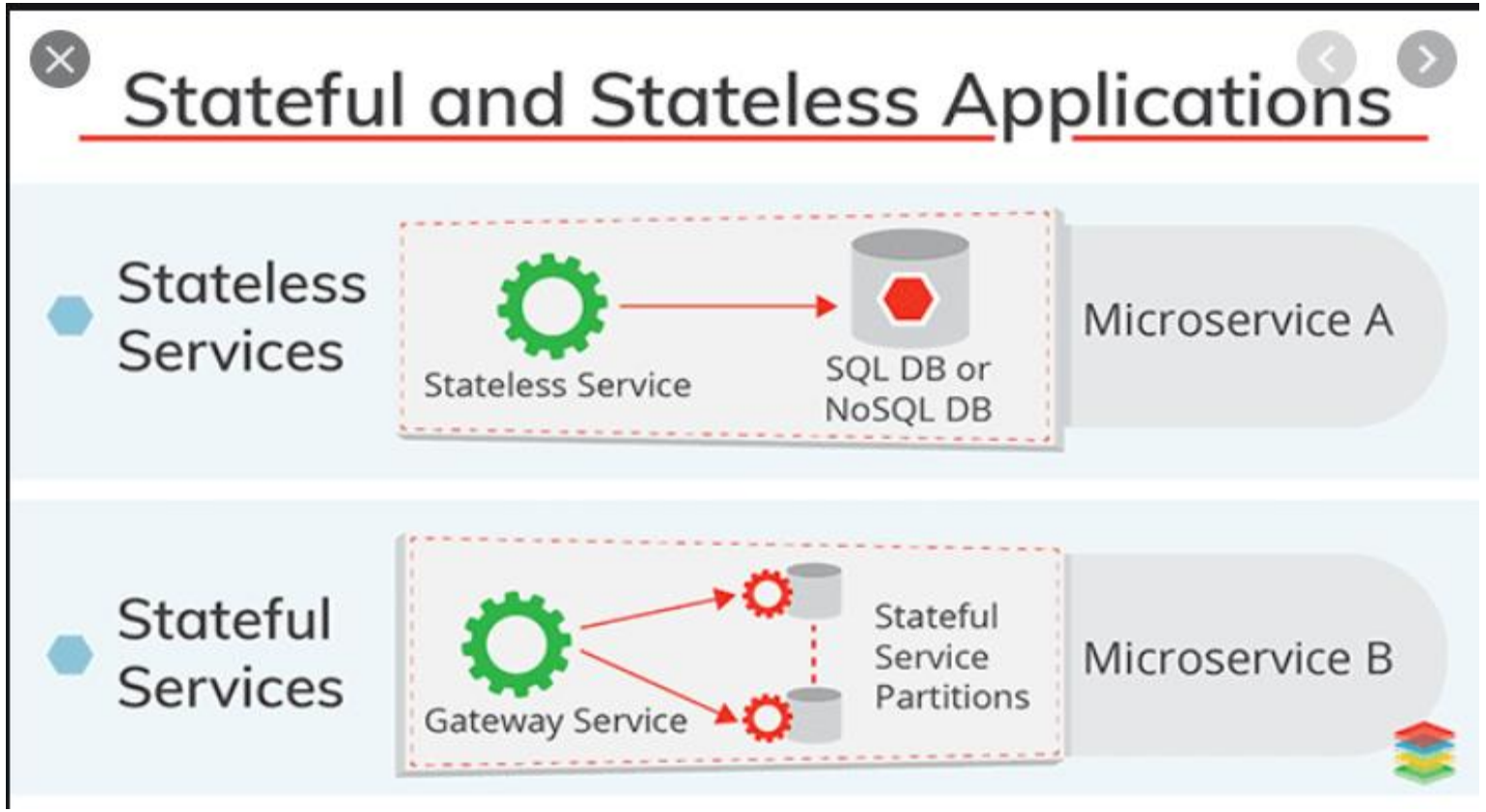
- In traditional approach designs, it is a common practice to process a **request and return immediately.**
- This limits the scalability of the application.
- By using Asynchronous communication, it is possible to add capacity by **adding additional servers when the application load increases.**



Design Considerations for Cloud Applications

3. Stateless Design

- **Stateless design** that store state outside of the components in a **separate database** allow scaling the application components independently.



Design Considerations for Cloud Applications

4. Database choice and design

- Choice of the database and the design of data storage schemes affect the application scalability.
- Decisions such as whether to choose a traditional relational database with strict schemes or a schema less database should be made after careful analysis of the applications data storage and analysis requirements.

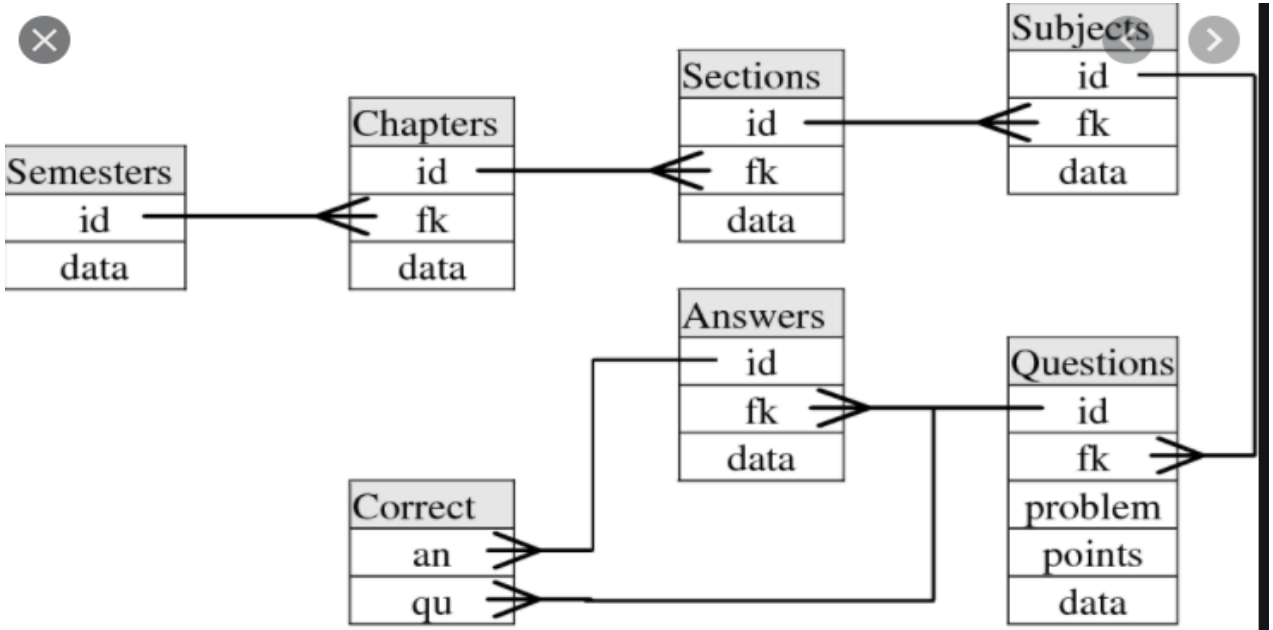


Figure 1. General Chemistry (1) relational database structure

Design Considerations for Cloud Applications

2. Reliability and Availability

- **Reliability** of a system is defined as the probability that a system will perform the intended functions under stated conditions for a specified amount of time.
- **Availability** is the probability that a system will perform a specified function under given conditions at a prescribed time.

Availability & Reliability

- What is availability ?
  - The degree to which a system, subsystem, or equipment is in a specified operable and committable state at the start of a mission, when the mission is called for at an unknown time.
  - Cloud system usually require high availability
    - Ex. "Five Nines" system would statistically provide 99.999% availability
- What is reliability ?
  - The ability of a system or component to perform its required functions under stated conditions for a specified period of time.
- But how to achieve these properties ?
  - Fault tolerance system
  - Require system resilience
  - Reliable system security

Design Considerations for Cloud Applications

- Reliability of a system is defined as the probability that a system will perform the intended functions **under stated conditions for a specified amount of time.**
- **Availability is the probability that a system will perform a specified function under given conditions at a prescribed time.**
- Important things to implement Reliable and available systems are :

1 . No single point of Failure

- **Traditional application design approaches which have single point of failure,** such as a **single database server or single application** server have the risk of complete breakdowns in case of the failure of the critical resource.
- To high achieve reliability and availability, **having a redundant(no longer needed)** resource or an automated fallback resource is important.

2. Trigger automated actions on failure

- Traditional application design approaches **handled failures by giving exceptions.**
- By using failures and triggers for automated actions it is **possible to improve the application reliability and availability.**

3. Graceful degradation

- Applications should be designed to gracefully degrade in the event of outages of some parts or components of the application.
- Graceful degradation means that if some component of the application becomes **unavailable,** the application as a whole would still be available and continue to serve the users, through with limited functionality.

Ex : e-commerce applications

4. Logging

- Logging all events in all the application components can help in detecting bottlenecks and failure so that necessary design/development changes can be made to improve application reliability and availability.

5. Replication (reproducing something)

- All application should be replicated.
- Replication is used to **create and maintain multiple copies of data in the cloud.**
- In the event of data loss at the primary location, organizations can continue to operate their applications from secondary data sources.

3. Security

- Security is an important design consideration for cloud applications given the outsourced nature of cloud computing environments.
- Key security considerations for cloud computing environments include :
  - Securing data at rest
  - Securing data in motion
  - Authentication
  - Authorization
  - Identity and access Management
  - Key Management
  - Data integrity
  - Auditing

4. Maintenance and Up gradation

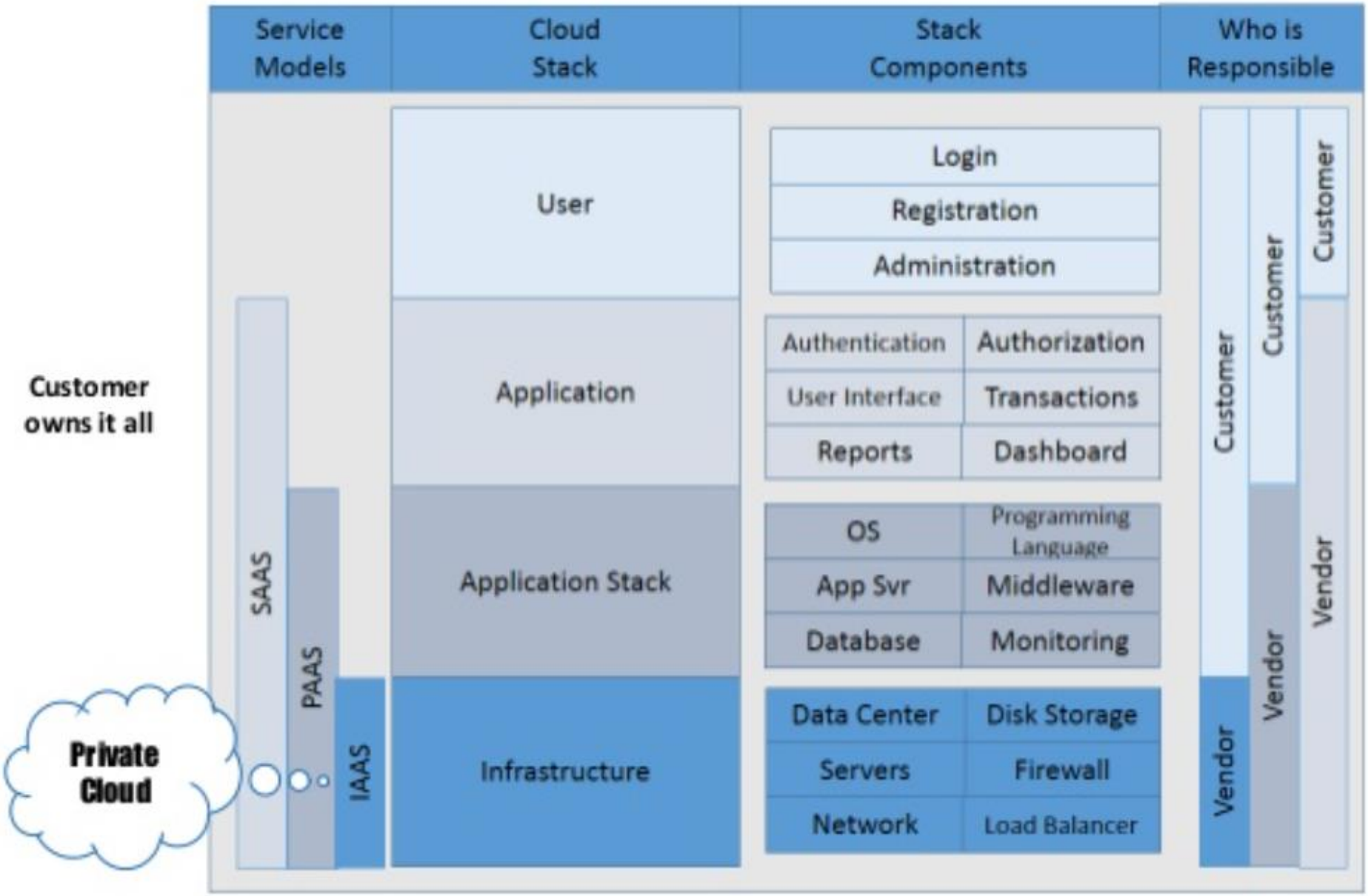
- To achieve a rapid time-to-market, businesses typically launch their applications with a core set of features ready and then incrementally add new features as and when they are complete.
- Businesses may need to adapt their applications based on the feedback from the users.
- In such scenarios, it is important to design applications with low maintenance and up gradation costs.
- Design decisions such as loosely coupled components help in reducing the application maintenance and up gradation time.
- In applications with loosely coupled components, changes can be made to a component with out affecting other components.
- Components can be tested individually .
- Other decisions such as logging and triggering automated actions also help in lowering the maintenance cost.

5. Performance

- Applications should be designed while keeping the performance requirements in mind.
- Performance requirements depend on the type of the application.

Ex: Applications which experience high database read-intensive workloads, can benefit from read – replication or caching approaches.

- There are various metrics that are used to evaluate the application performance like response time, throughput etc.


Service Models
Cloud Stack
Stack Components
Who is Responsible
Customer owns it all
SAAS
PAAS
IAAS
Private Cloud
User
Application
Application Stack
Infrastructure
Login
Registration
Administration
Authentication
Authorization
User Interface
Transactions
Reports
Dashboard
OS
Programming Language
App Svr
Middleware
Database
Monitoring
Data Center
Disk Storage
Servers
Firewall
Network
Load Balancer
Customer
Customer
Customer
Vendor
Vendor
Vendor

Reference Architectures for Cloud Applications

- Choosing the right deployment architecture is important to ensure that the application meets the specified performance requirements.
- Deployment architectures are used to build applications like e-commerce, business-to-business, Banking and Financial Applications.

1. Load Balancing Tier:

- Load balancing tier consists of one or more load balancers.
- It is recommended to have at least two load balancer instances to avoid the single point of failure.
- Whenever possible, it is also recommended to provision the load balancer instances in separate availability zones of the cloud service provider to improve reliability and availability.

Reference Architectures for Cloud Applications

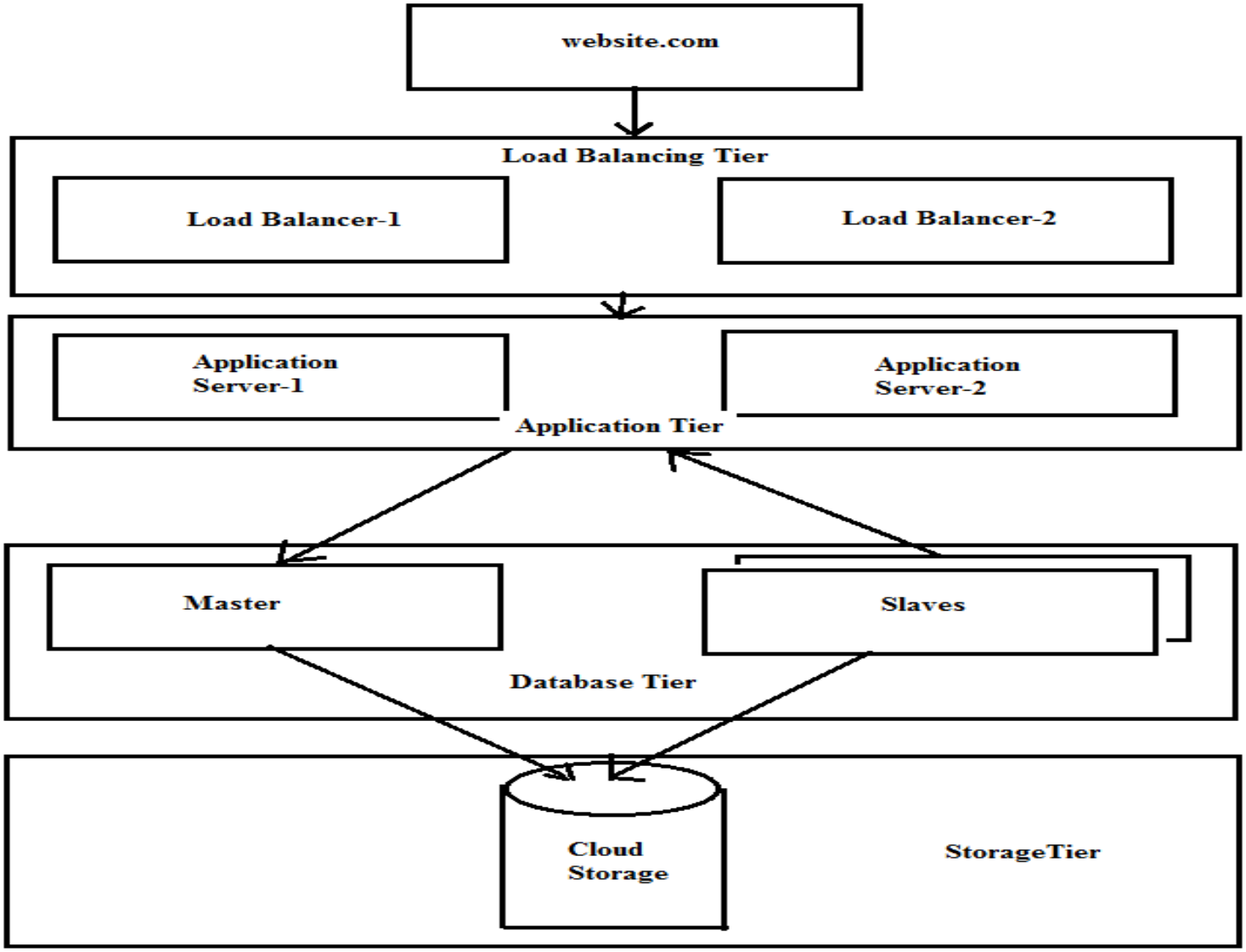


Fig : Deployment Architecture

2. Application Tier :

- Application tier is a second layer and it contains one or more application servers.
- For application tier it is recommended to configure auto scaling.
- Auto scaling can be triggered when the recorded values for any of the specific metrics such CPU usage, memory usage etc.. goes above defined thresholds.
- The minimum and maximum size of the application server auto scaling groups can be configured.
- It is recommended to have at least two application servers running at all times to avoid a single point of failure.
- If new instances are created when a auto scaling event is occurred and it may take few minutes for the instance to get fully operational.
- In this time period if the workload increases rapidly, the existing application server instances may fail to serve all requests.

3. Database tier:

- The third tier is database tier which includes a master database instance and multiple slave instances.
- The master node serves all the write requests and the read requests are served from the slave nodes.
- This improves the throughput for the database tier since most applications have a higher number of read requests than write requests.
- Multiple slave nodes also serve as a backup for the master node.
- In the case of failure of master node one of the slave nodes can be automatically configured to become the master.

4. Storage Tier:

- For both master and slave nodes it is highly recommended to use a disk sub system for storage and not the instance-attached store.
- This is important to ensure reliability and availability because in the event of failure if the instance-attached storage is used for the database, all data will be lost.
- In case of separate disk volumes, it is possible to restore the database.
- Regular snapshots of the database is recommended and the frequency of snapshots may be configured to be daily or hourly.
- It is recommended to store snapshots in distributed persistent cloud storage solutions like Amazon S3.

Example of Cloud Deployment tools:

Java:

1. Apache Tomcat 2. Oracle WebLogic 3. Glass Fish 4. IBM WebSphere 5. Jboss 6. ColdFusion 7. Apache Geronimo 8.Orion

PHP:

1. Zend Server 2. Quercus

.Net :

1. Internet Information Services(IIS) web server
2. Windows Server 3. App Fabric

Python

1. Django 2. Gunicorn 3. mod_python 4. mod_wsgi 5. paste

6. Tornado 7. Zope

Cloud Application Design Methodologies

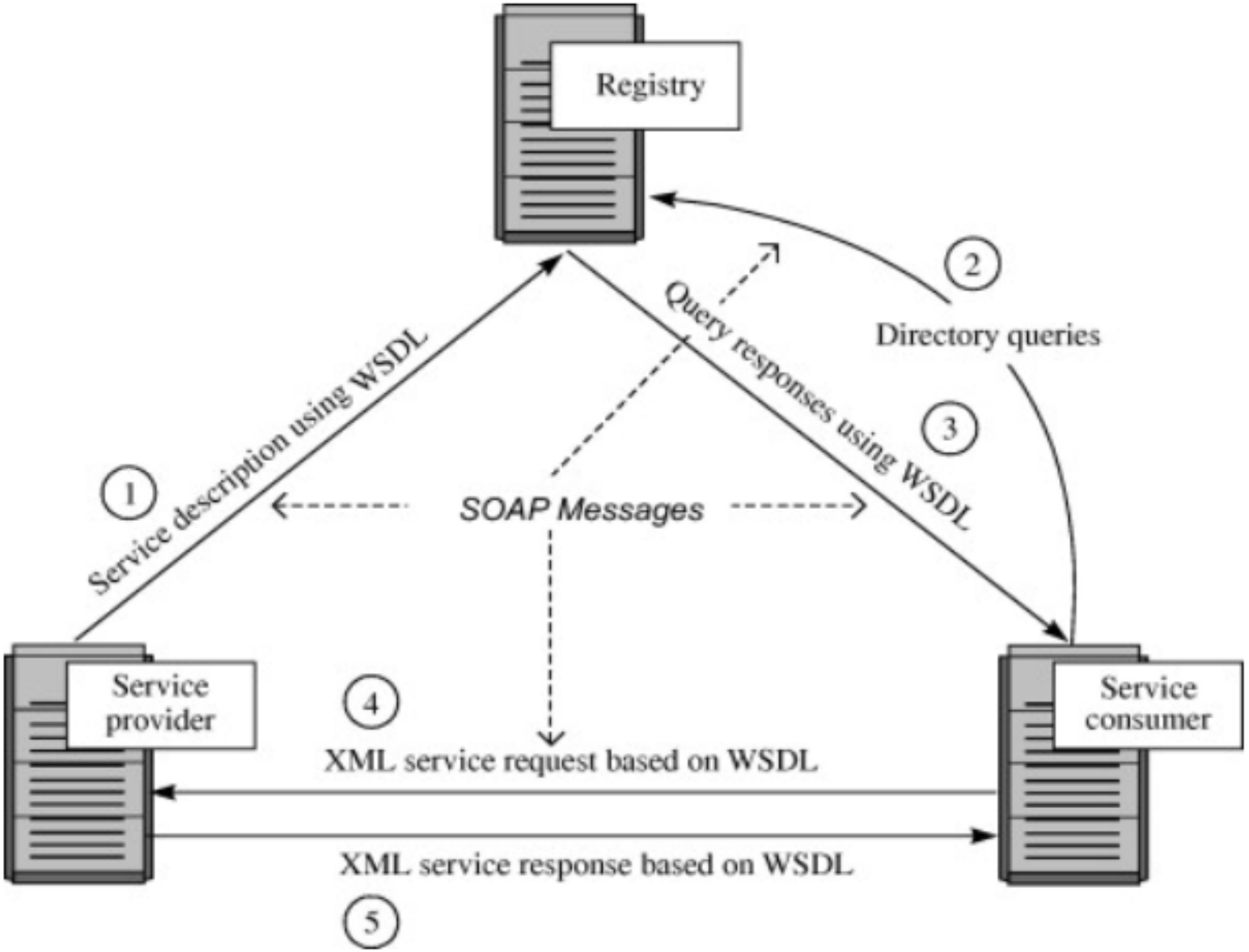
1. Service Oriented Architecture(SOA)

- It is a well established architectural approach for designing and developing applications in the form of services that can be shared and reused.
- SOA is a collection of discrete software modules or services that form a part of an application and collectively provide the functionality of an application.
- SOA services are developed as loosely coupled modules with no hardwired calls embedded in the services.
- The services communicate with each other by passing messages.
- Services are described using the Web Services Description Language(WSDL).
- WSDL is an XML-based web services description language that is used to create services descriptions containing information on the

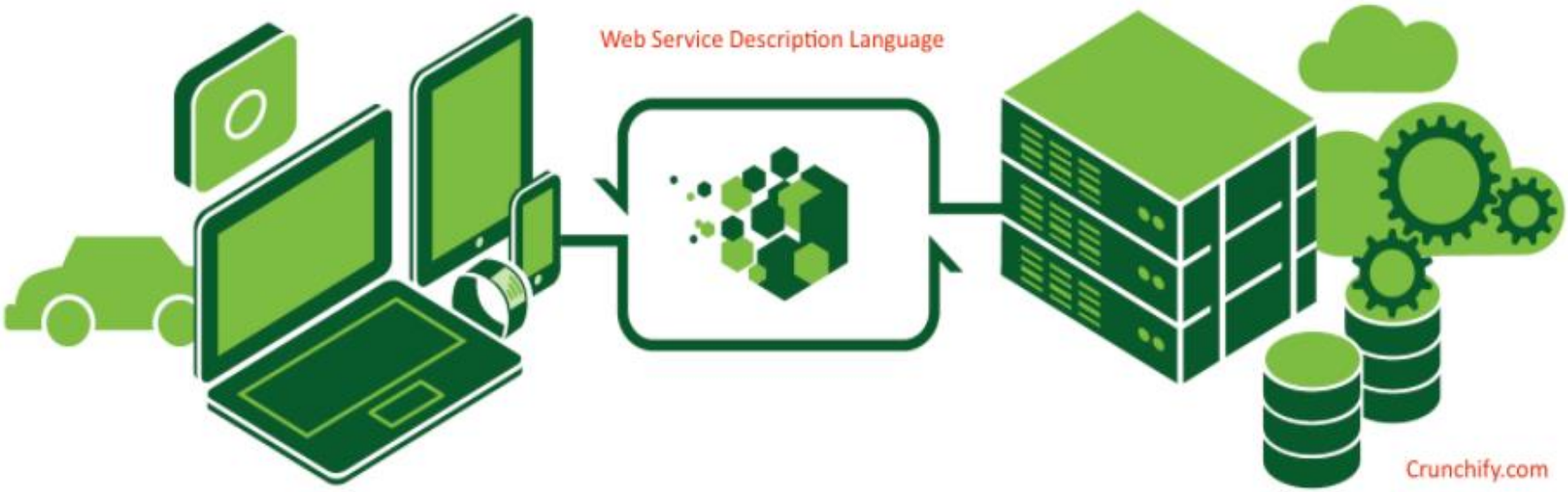
Cloud Application Design Methodologies

- Concepts of WSDL (Web Services Description Language)

1. Services: Services describes a discrete system function that is exposed as a web service.
2. Endpoint: Endpoint is the address of the web service.
3. Binding: It specifies the interface and transport protocol.
4. Interface: Interface defines a web service and the operations that can be performed by the service and the input and outputs.
5. Operation: Operation defines how the message is decoded and the actions that can be performed.
6. Types: Types describe the data.



Simple Object Access Protocol(SOAP) Web Services Description Language (WSDL)



The Web Services Description Language (WSDL) is an XML-based language that is used for describing the functionality offered by a Web service

Cloud Application Design Methodologies

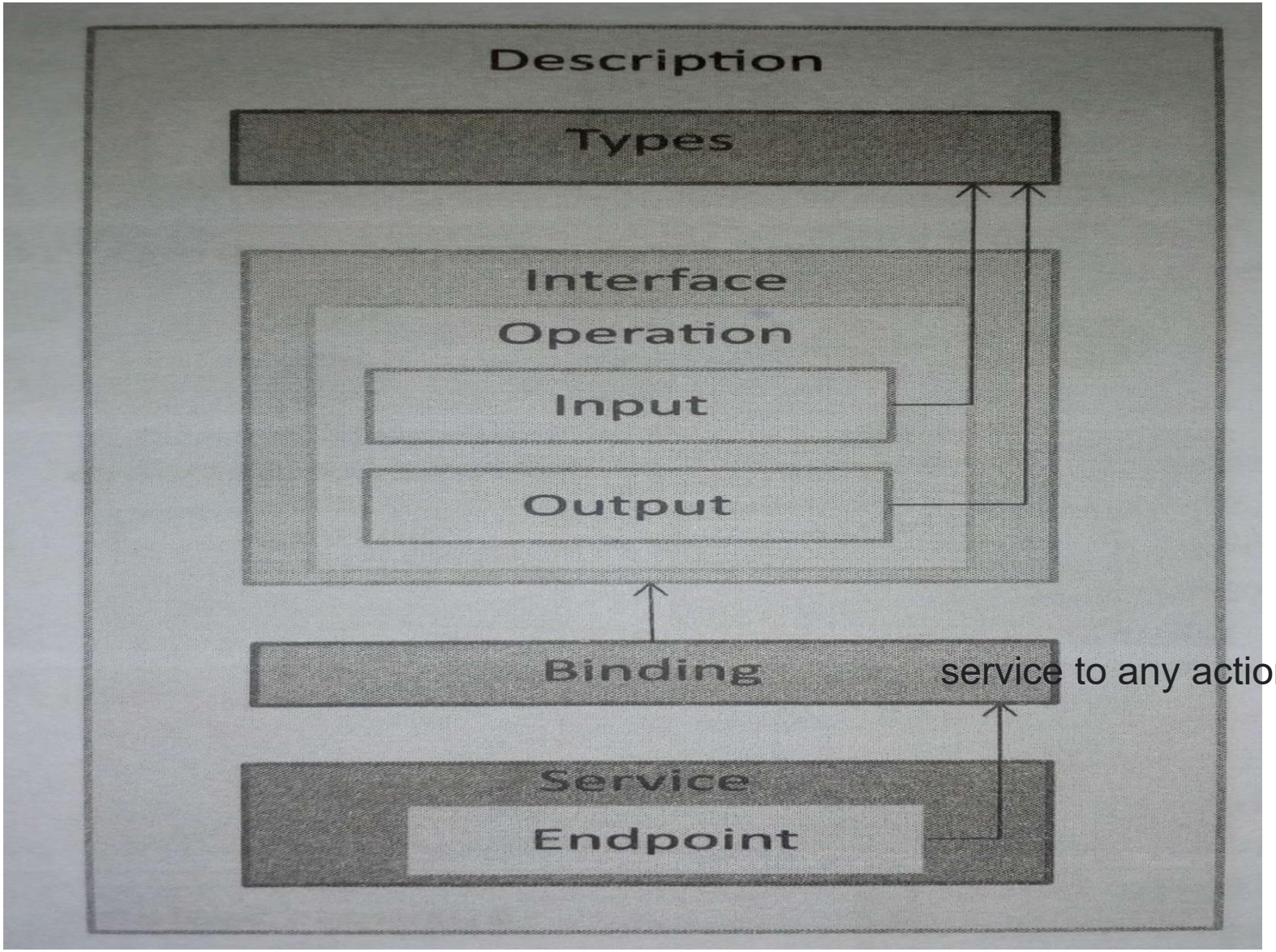


Fig : WSDL concepts for representation of web services

- SOA services communicate using the **Simple Object Oriented Protocol(SOAP).**
- SOAP is a protocol that allows exchange of structured information between web services.
- WSDL in combination with SOAP is used to provide web services over the internet.
- SOA allows reuse of services for multiple applications.

Layers of SOA :

Business Systems:

- This layer consists of custom built applications and legacy systems(*outdated computing software*) such as Enterprise Resource Planning(ERP), Custom Relationship Management(CRM), Supply Chain Management(SCM) etc.

Service Components:

- It allow the layers above to interact with the business systems. These are responsible for realizing the functionality of the services exposed.

Composite Services:

- These are course-grained services which are composed of two or more service components.
- Composite services can be used to create enterprise scale components or business-unit specific components.

Orchestrated Business Processes :

- Composite services can be orchestrated to create higher level business processes.
- In this layers the compositions and orchestrations of the composite services are defined to create business processes.

Presentation Services:

- This is the topmost layer that includes user interfaces that exposes the services and the orchestrated business processes to the users.

Enterprise Service Bus:

- This layer integrates the services through adapters, routing, transformation and messaging mechanisms.

Cloud Application Design Methodologies

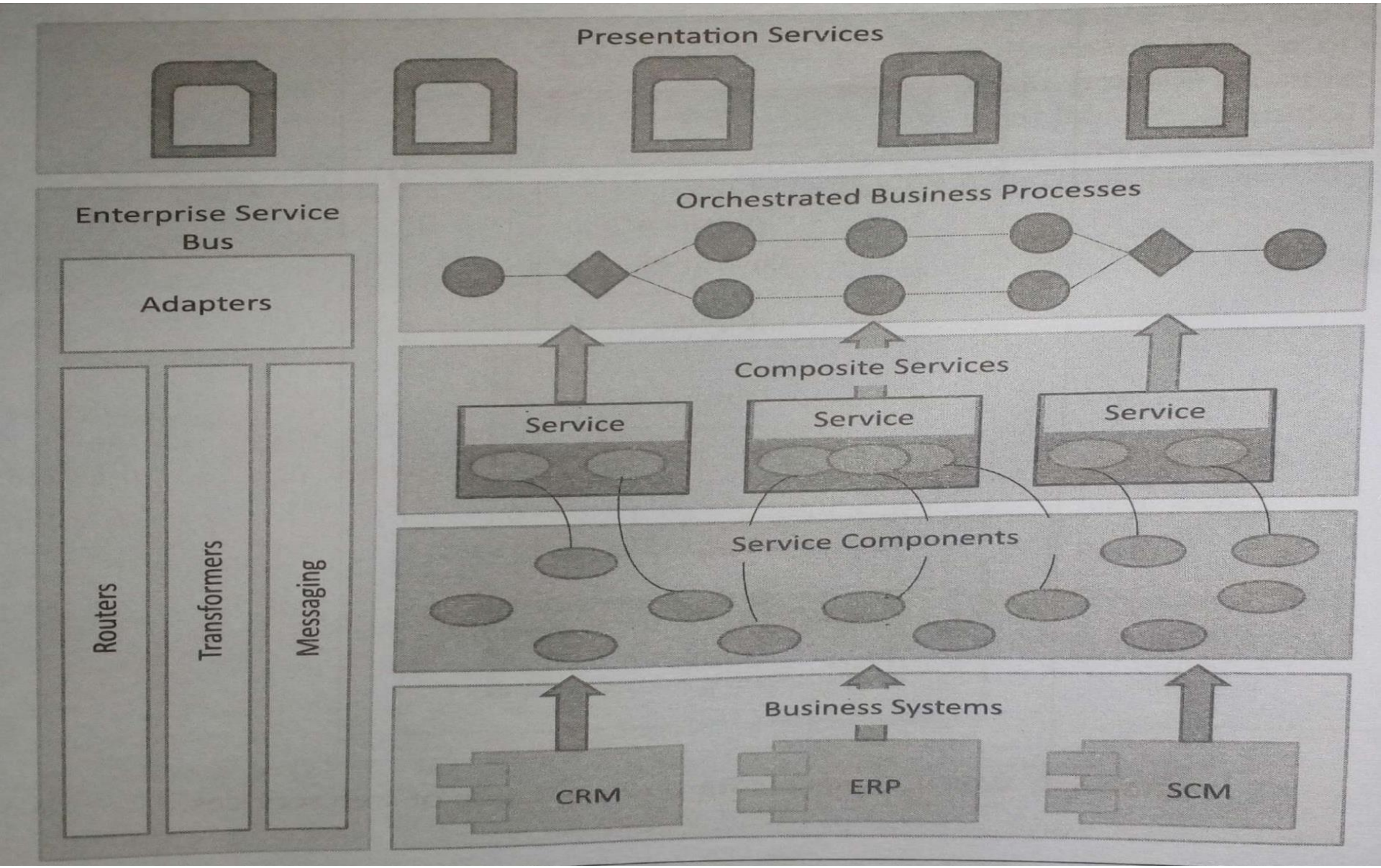


Fig : Layers of service oriented architecture

2. Cloud Component Model (CCM)

- It is an Application design methodology that provides a flexible way of creating cloud applications in a rapid, convenient and platform independent manner.
- CCM is an architectural approach for cloud applications that is not tied to any specific programming language or cloud platform.
- Applications designed using CCM have better portability and interoperability.
- CCM based applications have better scalability by decoupling application components and providing asynchronous communication mechanism.
- CCM makes the application individual components upgraded independent of other components.
- Cost benefits come by scaling cloud resources up or scaling out only for those components which require additional computing capacity.

Cloud Component Model (CCM)

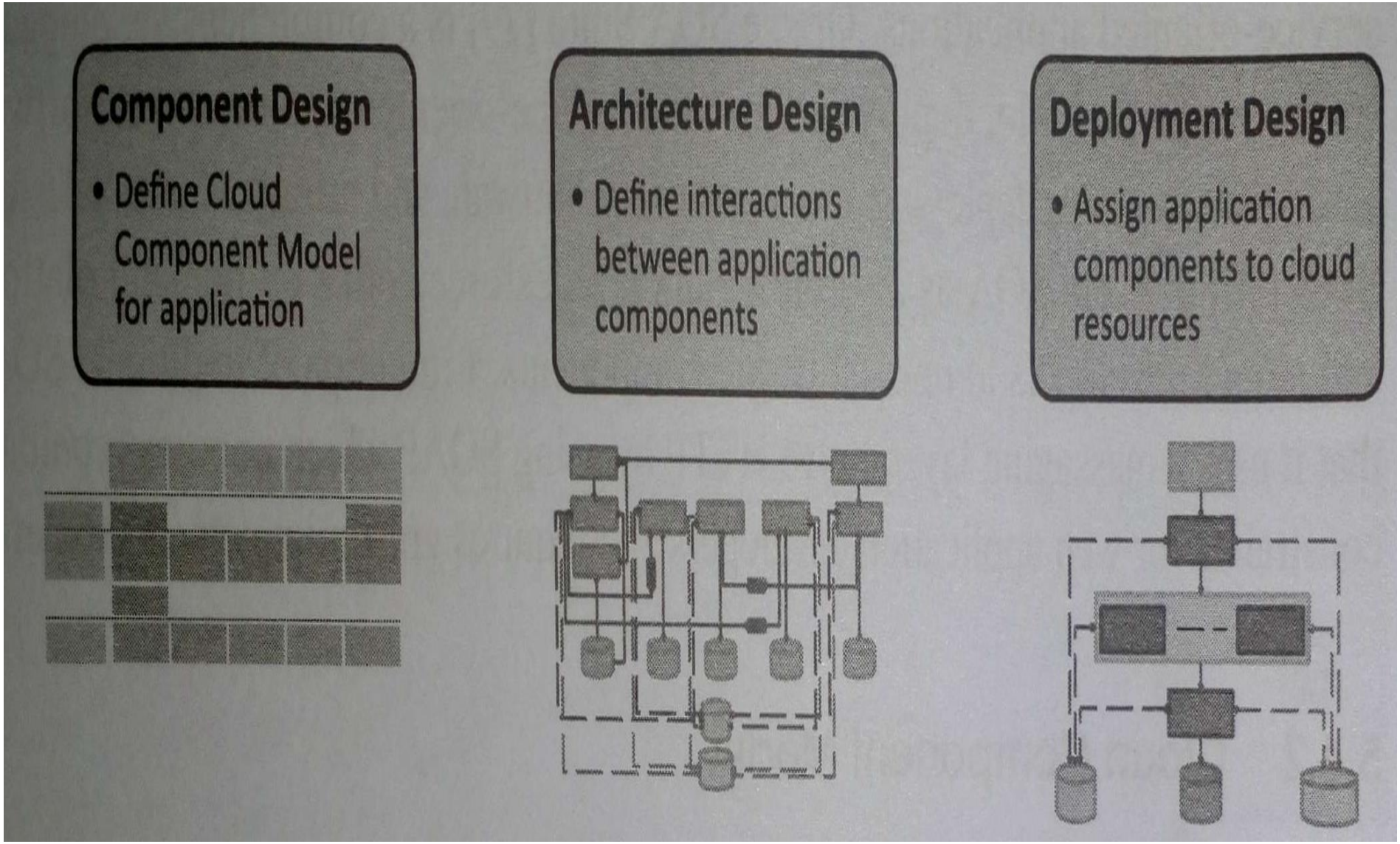


Fig : CCM model Design Steps

A. Component Design

- CCM is created for the application based on comprehensive analysis of the applications functions and building blocks.
- CCM allows identifying the building blocks of a cloud application which are classified based on the functions performed and type of cloud resources required.

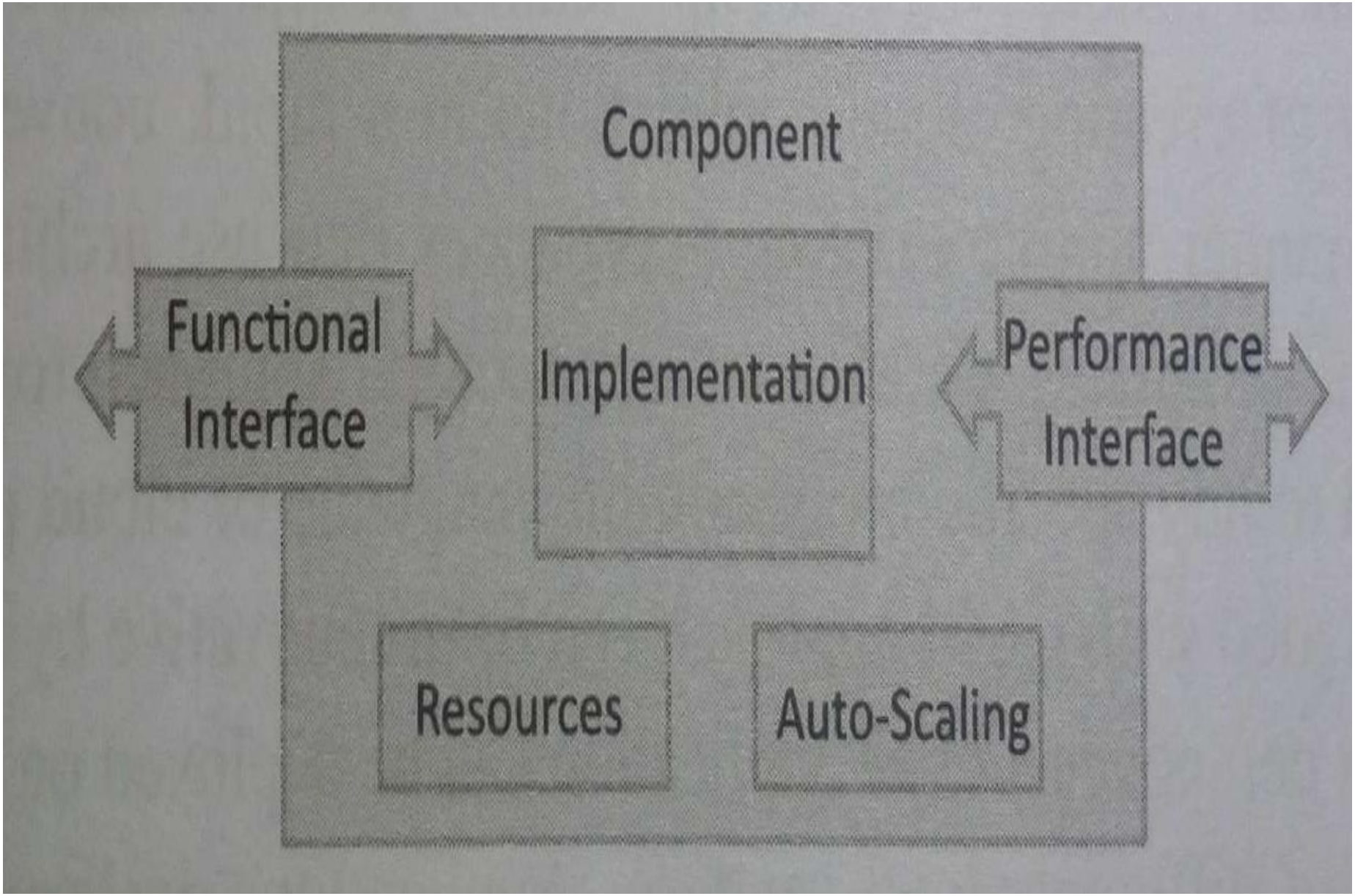


Fig : Architecture of CCM component

B. Architecture Design

- In the Architecture design interactions between the application components are defined.

Characteristics of CCM components:

1. Loose Coupling :

- Components in the CCM are loosely coupled.
- Instead of hard-writing the links, the components interface through clearly defined functional and service boundaries.
- Loose coupling of components relies on the use of REST communication protocol that allows components developed in different programming languages to communicate with each other.

2. Asynchronous communication:

- Loosely coupled components communicate asynchronously through message based communication.
- It isolates all components so all components interact asynchronously by treating other components as black box.

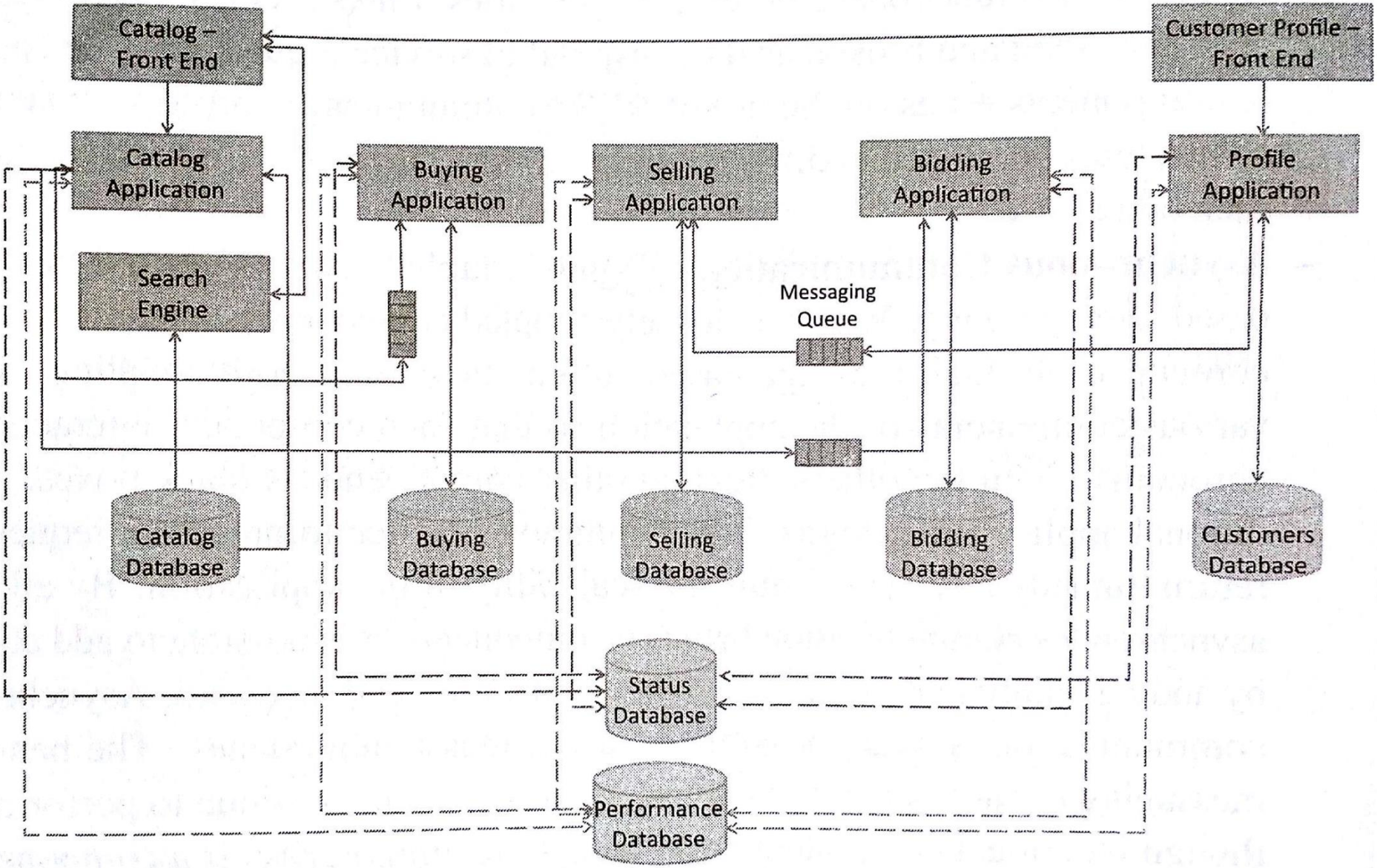


Fig : Architecture design of e-commerce application

3.Stateles Design:

- CCM components are stateless.
- By storing session state outside of the component, stateless component design enables distribution and horizontal scaling.
- In distributed computing successive requests to a component may be serviced by different servers.

C. Deployment Design

- In this application components are mapped to specific cloud resources such as web servers, database servers.
- All application components are designed loosely coupled and asynchronous communication, components are deployed independently of each other.
- Multiple clouds can be used for application deployment.
- This approach makes it easy to migrate application components from one cloud to another.
- The application developer can ensure that the applications meet the performance and cost requirements with the changing context.

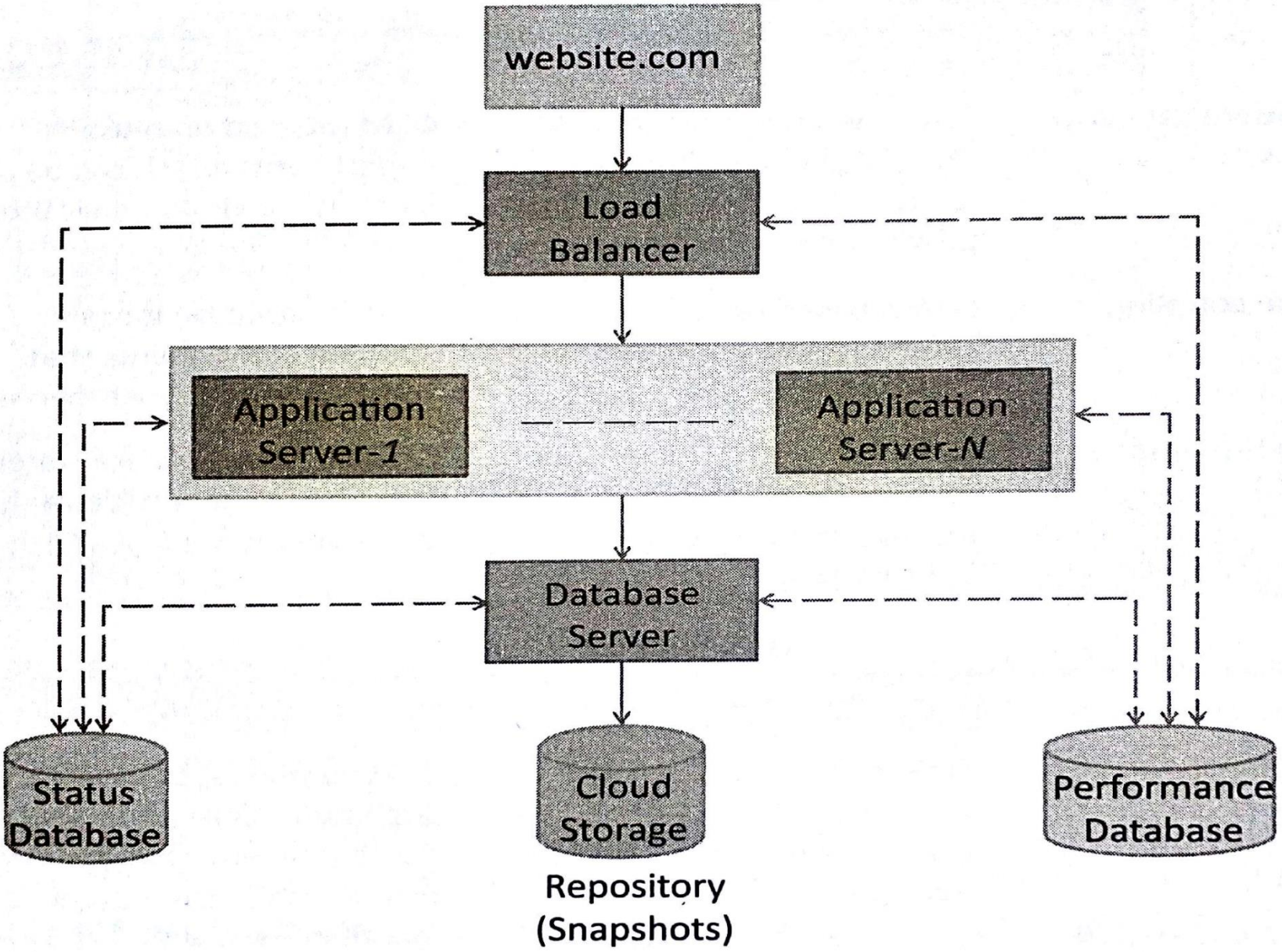


Fig : Deployment Design steps for an e-commerce application

3. Iaas, Paas and SaaS services for cloud applications

- Cloud services provides such as Amazon, google, Microsoft etc provide diverse infrastructure(IaaS), platform(PaaS) and software (SaaS) that the developers can use for developing and deploying applications in cloud environments.
- ***cloud service provider (***CSP) make cloud resources avilable to the users with cloud APIs via a web based interface.
- There are different CSPs to provide various cloud resources with their APIs.
- Ex : Salesforce Paas it allows developers to create applications using Apex.

4. Model View Controller (MVC):

- MVC is a popular software design pattern for web application.
- It consists of three parts :

1. Model

- Model manages the data and the behavior of the applications.
- Model processes events sent by the controller.
- Model has no information about the views and controllers.
- Model responds to the requests for information about its state(for the view) and responds to the instructions to change state(from controller).

2. View

- View prepares the interface which is shown to the user.
- Users interact with the application through views.

3. Controller

- Controller glues(serverless data integration service) the model to the view.
- Controller processes user requests and updates the model when the user manipulates the view.
- Controller also updates the view when the model changes.

Benefits :

- Main advantage of using MVC is that improves the maintainability of the application and allows reuse of code.
- Easy to update due to the separation of the model from the view.
- Model does not depend on view and controller, so it allows the model to be developed and tested independently.

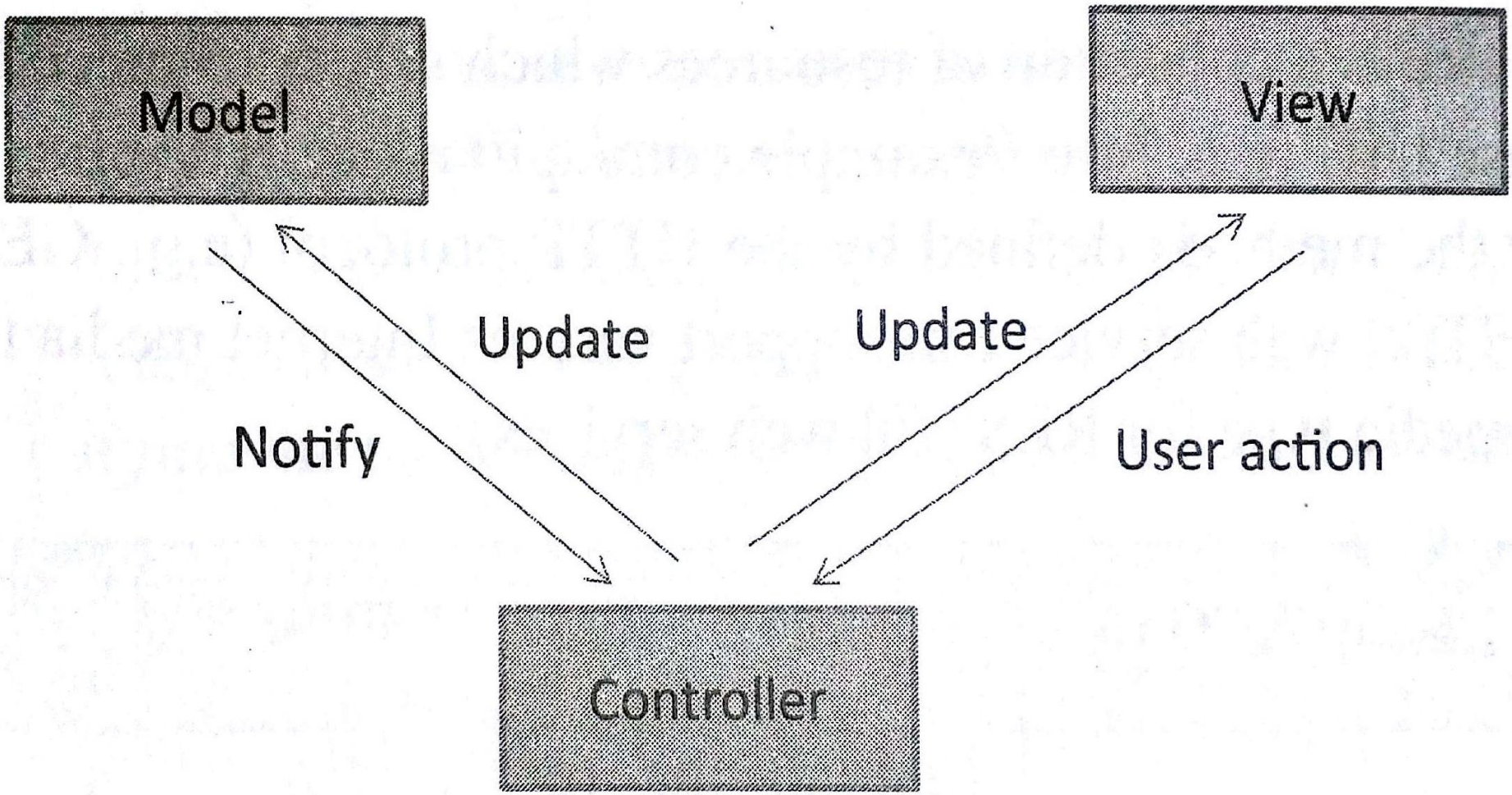


Fig : Model View Controller

Cloud Application Design Methodologies

REST

- Representational State Transfer (REST) is a set of architectural principles by which you can design web services and web APIs that focus on a systems's resources and how resource states are addressed and transferred.
- The REST architectural constraints apply to the components, connectors, data elements with a distributed hypermedia systems.

Constraints are :

1. Client-Server :

- The principle behind the client-server constraint is the separation of concerns.
- Example clients should not be concerned with the storage of data which is a concern of the server.
- Same like server should not be concerned about the user intreface, which is concern of the client.
- Separation allows client and server to be independently developed.

RESTful Web Services

2. Stateless :

- Each request from client to server must contain all of the information necessary to understand the request and cannot take advantage of any stored context on the server.
- The session state is kept entirely on the client.

3. Cacheable :

- Cache constraint requires that the data with in a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable.
- If a response is a cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
- Caching can partially or completely eliminate some interactions and improve efficiency and scalability.

RESTful Web Services

4. Layered System :

- Layered system constraint constraints the behavior of components such that each component cannot see beyond the immediate layer with which they are interacting.
- Ex : A client can not tell whether it is connected directly to the end server or to an intermediary along the way.
- System scalability can be improved by allowing intermediaries to respond to requests instead of the end server, without the client having to do anything different.

5. Uniform Interface:

- This constraint requires that the method of communication between a client and a server must be uniform.
- Resources are identified in the requests and are themselves separate from the representations of the resources that returned to the client.
- When a client holds a representation of a resource it has all the information required to update or delete the resource.

RESTful Web Services

6. Code on demand:

- Servers can provide executable code or scripts for clients to execute in their context.
- This constraint is the only one that is optional.

- A RESTful web service is a web API implemented using HTTP and REST principles.
- RESTful web service is a collection of resources which are represented by URIs.
- The client send requests to these URIs using the methods defined by the HTTP protocol.
- A RESTful web service can support various internet media types .

Data Storage Approaches

- There are **two** main categories of database approaches

1. Relational(SQL) Approach 2. Non – Relational(No-SQL) Approach

Relational(SQL) Approach

- key that uniquely identifies each tuple in the relation.
- A relational database is database that conforms to the relational model.
- A relational database is a collection of relations or tables.
- A relation is a set of tuples (**A single entry in a table**) or rows.
- Each relation has a fixed schema that defines the set of attributes or columns in a table and the constrains on the attributes.
- Each tuple in a relation has the same attributes.
- The tuples in a relation can have any order
- Each attribute has domain, which is the set of possible values for the attribute.
- Relations can be modified using insert, update and delete operations.
- Every relation has a primary

Database Constraint :

1. Domain Constraint

- Domain constraint restrict the domain of each attribute or set of possible values for the attribute.
- Domain constraints specify that each in tuple (**A single entry in a table**), the value of each attribute must be a value from the domain of the attribute.

2. Entity Integrity Constraint

- It states that no primary key value can be null.
- Since primary key is used to uniquely identify each tuple in a relation, having null value for a primary key value will make it impossible to identify tuples in the relation.

3. Referential Integrity Constraint

- These are required to maintain consistency among the tuples in two relations.
- Referential integrity requires every value of one attribute of a relation to exists as a value of another attribute in another relation.

4. Foreign Key

- For cross-referencing between multiple relations foreign keys are used.
- Foreign key is a key in a relation that matches the primary key of another relation.

- To provide reliable transactions Relational database provide **Atomicity, Consistency, Isolation, Durability** (ACID) properties :

1. Atomicity :

- Atomicity property ensures that each transaction is either "all or nothing" .
- An atomicity transaction ensures that all parts of the transaction complete or the database state is left unchanged.

2. Consistency:

- Consistency ensures that each transaction brings the database from one valid state to another.
- The data in a database always conforms to the defined schema and constraints.

3. Isolation :

- Isolation property ensures that the database state obtained after a set of concurrent transactions is the same as would have been if the transactions were executed serially.
- **The results of incomplete transactions are not visible to other transactions.**
- The transactions are isolated from each other until they finish.

4. Durability :

- Durability property ensures that once a transaction is committed, the data remains as it is, i.e it is not affected by system outages such as power loss.
- Durability guarantees that the database can keep track of changes and can recover from abnormal terminations.

Data Storage Approaches

Pros	Cons
Well defined consistent model. An application that runs on one relational database (such as MySQL) can be easily changed to run on other relational databases (eg. Microsoft SQL server). The underlying model remains unchanged.	Performance is the major constraint for relational databases. The performance depends on the number of relations and the size of the relations. Scaling out relational database deployments is difficult.
Provide ACID guarantees.	Limited support for complex data structures. Eg. If the data is naturally organized in a hierarchical manner and stored as such, the hierarchical approach can allow quick analysis of data.
Relational integrity maintained through entity and referential integrity constraints.	A complete knowledge of the database structure is required to create ad hoc queries.
Well suited for Online Transaction Processing (OLTP) applications.	Most relation database systems are expensive.
Sound theoretical foundation (based on relational model) which has been tried and tested for several years. Stable and standardized databases available.	Some relational databases have limits on the size of the fields.
The database design and normalization steps are well defined and the underlying structure is well understood.	Integrating data from multiple relational database systems can be cumbersome.

2. Non – Relational(No-SQL) Approach :

- Non- relational databases are becoming popular with the growth of cloud computing.
- Non-relational databases have better horizontal scaling capability and improved performance for big data at the cost of less rigorous consistency models.
- Non relational databases do not provide ACID guarantees.
- The driving force behind the non-relational databases is the need for databases that can achieve high scalability, fault tolerance and availability.
- These databases can be distributed on a large cluster of machines.
- Fault tolerance is provided by storing multiple replicas of data on different machines.
- These systems are optimized for fast retrieval and appending operations on records.

- Non-relational databases are popular for applications in which the scale of data involved is massive, the data may not be structured and real time performance is important as opposed to consistency.
- It does not have a strict schema.

General Categories of Records :

1. Key-value store :

- Key-value store databases are suited for applications that require storing unstructured data without a fixed schema.
- Most key value stores have support for native programming language data types.

2. Document Store :

- Document store databases store semi-structure data in the form of documents which are encoded in different standards like JSON,XML,BSON,YAML etc.

Data Storage Approaches

3. Graph Store :

- Graph store are designed for storing data that has graph structure.
- These solutions are suitable for applications that involve graph data such as social networks, transportation system etc.

4. Object Store :

- Object store solutions are designed for storing data in the form of objects defined in an object-oriented programming languages.

Data Storage Approaches

Pros	Cons
Easy to scale-out. Higher performance for massive scale data as compared to relational databases. Allows sharing of data across multiple servers.	Do not provide ACID guarantees, therefore less suitable for applications such as transaction processing that require strong consistency.
Most solutions are either open-source or cheaper as compared to relational databases.	No fixed schema. There is no common data storage model. Different solutions have different data storage models.
High availability and fault tolerance provided by data replication.	Limited support for aggregation (SUM, AVG, COUNT, GROUP BY) as compared to relational databases.
Support complex data structures and native programming objects.	Performance for complex joins is poor as compared to relational databases.
No fixed schema. Support unstructured data.	No well defined approach for database design, since different solutions have different data storage models.
Very fast retrieval of data. Suitable for real-time applications.	Lack of a consistent model can lead to solution lock-in, i.e., migrating from one solution to other may require significant remodeling of the application.
Most solutions provide support for MapReduce programming model for processing massive scale data.	

Thank you